

Data Communication between Smartphone and Raspberry Pi over Bluetooth

Step 1: Configure Bluetooth on Raspberry Pi

Before writing code, configure the Pi's Bluetooth daemon to enable Serial Port Profile (SPP) and pair your phone.

1. **Enable Compatibility Mode (SPP)** Open the systemd Bluetooth configuration file:

```
sudo nano /etc/systemd/system/dbus-org.bluez.service
```

Find the line starting with `ExecStart=/usr/libexec/bluetooth/bluetoothd` and append `-C` (or `--compat`) to the end of that line:

```
ExecStart=/usr/libexec/bluetooth/bluetoothd -C
```

Directly beneath that line, add this row to add the Serial profile:

```
ExecStartPost=/usr/bin/sdptool add SP
```

Save and exit (`Ctrl+O` , `Enter` , `Ctrl+X`).

2. **Restart Bluetooth Service**

```
sudo systemctl daemon-reload  
sudo systemctl restart bluetooth
```

3. **Set Proper Permissions for Bluetooth Socket**

```
sudo chmod 777 /var/run/sdp
```

4. **Pair Smartphone with Raspberry Pi** Run `bluetoothctl` to pair your phone:

```
sudo bluetoothctl
```

Inside the `bluetoothctl` prompt, run:

```
discoverable on
pairable on
agent on
default-agent
```

- On your **Smartphone**, turn on Bluetooth, scan for devices, and select your Raspberry Pi.
- Accept the pairing code on both devices.
- Type `exit` to quit `bluetoothctl` .

Step 2: Install PyBluez Library

Install the required Python Bluetooth packages on your Raspberry Pi:

```
sudo apt update
sudo apt install python3-pip libbluetooth-dev build-essential -y
pip3 install pybluez
```

Step 3: Python Bluetooth Server Code (Raspberry Pi)

Create a Python script on the Raspberry Pi named `bt_server.py` . This script listens for incoming connections from your phone, reads data sent by the phone, and sends a reply back.

```
import socket

def run_bt_server():
    # RFCOMM Bluetooth socket creation
    server_sock = socket.socket(socket.AF_BLUETOOTH, socket.SOCK_STREAM, socket.BTPROTO_RFCOMM)

    # Port 1 is standard for RFCOMM channel 1
    port = 1

    try:
        # Bind socket to any local Bluetooth adapter
        server_sock.bind(("", port))
        server_sock.listen(1)
```

```

print(f"Bluetooth Server running... Waiting for connection on RFCOMM channel {port}")

# Accept client connection
client_sock, client_info = server_sock.accept()
print(f"Accepted connection from MAC Address: {client_info[0]}")

while True:
    # Receive up to 1024 bytes from phone
    data = client_sock.recv(1024)
    if not data:
        break

    decoded_msg = data.decode("utf-8").strip()
    print(f"Received from Phone: {decoded_msg}")

    # Send echo response back to Phone
    response = f"Pi Received: {decoded_msg}\n"
    client_sock.send(response.encode("utf-8"))

    # Stop loop if phone sends 'exit'
    if decoded_msg.lower() == "exit":
        print("Exit signal received. Closing connection.")
        break

except Exception as e:
    print(f"Error occurred: {e}")

finally:
    print("Closing sockets...")
    if 'client_sock' in locals():
        client_sock.close()
    server_sock.close()

if __name__ == "__main__":
    run_bt_server()

```

Step 4: Test Communication with Smartphone

1. Install a Bluetooth Terminal App on Smartphone

- **Android:** Download **Serial Bluetooth Terminal** (by Kai Morich) from Google Play Store.
- (Note: *iOS devices do not support classic Bluetooth RFCOMM without BLE protocols; an Android phone is recommended for standard SPP.*)

2. Connect App to Raspberry Pi

- Open **Serial Bluetooth Terminal** on your smartphone.
- Open the app menu $\rightarrow$ **Devices** $\rightarrow$ **Paired**.

- Select your **Raspberry Pi**.

3. Run Python Server on Raspberry Pi

Run the Python script in the terminal:

```
python3 bt_server.py
```

4. Exchange Data

- In the smartphone app, tap the **Connect** icon at the top right.
- Type a message (e.g., `Hello Pi`) and tap send.
- Check your Pi terminal—you will see the incoming message, and your phone screen will display `Pi Received: Hello Pi` .